



Riesgo de crédito de la banca privada del Ecuador

Enero - Agosto 2026

1. Contexto, pregunta y datos

El sistema financiero ecuatoriano opera en un entorno de estabilización macroeconómica, caracterizado por una mejora en la percepción de la deuda soberana. No obstante, se mantiene una alta sensibilidad regulatoria y climática debido al desarrollo del Fenómeno de El Niño.

Durante el periodo de análisis, la gestión del riesgo de crédito en la banca privada se fundamenta en tres pilares de datos clave:

A. Calidad de cartera y morosidad controlada

La morosidad de la banca privada muestra una tendencia a la baja en comparación con el año anterior, ubicándose en 3,05%. A pesar de esta contracción agregada, el indicador refleja una marcada heterogeneidad según las variables de distribución:

- Segmento de producto: La cartera se concentra fuertemente en el segmento productivo (47,4%) y consumo (40%). El segmento de vivienda, aunque minoritario, ha dinamizado su colocación debido a la reducción de tasas de interés durante el primer semestre.
- Distribución geográfica: Los niveles de mora difieren significativamente entre las regiones del país, acentuándose en las zonas con mayor exposición a actividades agrícolas y comerciales afectadas por factores estacionales.

B. Cobertura frente a impagos y provisiones

La banca privada ha fortalecido su resiliencia mediante niveles de reservas históricos. La tasa de cobertura de la cartera improductiva alcanzó un récord de 212%. Esta sólida posición de provisiones dota a las instituciones de la capacidad de absorción necesaria para resistir un eventual incremento en el índice de incumplimiento de hasta el 6% en el sector privado sin comprometer su solvencia.

C. Liquidez, fondeo y entorno de crédito

- Descalce de tasas y liquidez: Los depósitos del sistema alcanzaron los USD 63.480 millones (un crecimiento anual del 11,6%). Sin embargo, dado que la colocación de la cartera bruta evolucionó a un ritmo mayor, el indicador de liquidez general del sistema experimentó una leve contracción, situándose en 19,20%.
- Reformas al historial crediticio: En agosto de 2026, la Junta de Política y Regulación Financiera aprobó reformas que amplían las obligaciones financieras reportadas en el Registro de Datos Crediticios. Esto modifica las metodologías internas de los burós de crédito y obliga a recalibrar

Proyecto final - Dashboard EDA por profesión



las herramientas de *scoring* bancario para mitigar el sobreendeudamiento.

1.1 Profesión, problema, decisión y pregunta

La investigación se desarrolla en el campo de las finanzas corporativas, específicamente desde la perspectiva de un analista de riesgo de crédito en la banca privada del Ecuador.

El problema central radica en que la morosidad crediticia se distribuye de manera heterogénea, registrando variaciones significativas en función del segmento de producto financiero, la región geográfica y la entidad evaluada.

Este escenario se ve agravado durante el periodo de análisis por la incertidumbre macroeconómica y climática derivada del desarrollo proyectado del Fenómeno de El Niño (Banco Central del Ecuador, 2026).

La decisión estratégica que se pretende respaldar a través de este análisis técnico abarca dos frentes:

- Mitigación del riesgo: Identificar las carteras vulnerables para reforzar las provisiones específicas o endurecer las políticas de colocación crediticia.
- Optimización comercial: Detectar los nichos de mercado con bajo perfil de riesgo que presenten espacio óptimo para la expansión de la colocación.

A partir de este contexto, se plantea la siguiente pregunta central de investigación:

¿Qué segmentos de crédito, regiones geográficas y entidades financieras concentran el mayor riesgo de morosidad dentro del sistema bancario privado ecuatoriano?

2 Origen de los datos y cómo se obtuvieron

Los datos son *reales, no simulados*. Proviene de la Superintendencia de Bancos del Ecuador, Portal Estadístico, sección *Cartera y Depósitos*, carpeta *Año 2026 / CARTERA*. Se descargaron cuatro archivos Excel de banca privada con corte a agosto de 2026: consumo, productivo, microcrédito y vivienda (esta incluye inmobiliario y vivienda de interés público).

Cada Excel trae una hoja tipo reporte (con celdas combinadas) y una hoja BASE en formato tabla. El proyecto usa solo las hojas BASE. El resultado es una tabla de 13.765 filas con 24 entidades, 25 provincias y 122 cantones, mes a mes de enero a agosto.

2.1 Origen de los datos y metodología de extracción

Los datos empleados en este estudio son de naturaleza empírica e institucional. La información



Programación y Análisis de Datos (MIACD02P01)

macroeconómica y sectorial proviene del portal estadístico de la Superintendencia de Bancos del Ecuador (SB, 2026), específicamente de la sección de Cartera y Depósitos, dentro del repositorio correspondiente al año fiscal 2026.

Para la construcción de la base de datos de la investigación, se descargaron cuatro archivos en formato de hoja de cálculo (Excel) con corte a agosto de 2026, correspondientes a las estructuras de los principales portafolios de la banca privada:

- Consumo
- Productivo
- Microcrédito
- Vivienda (incluye los segmentos inmobiliario y vivienda de interés público)

Cada archivo de origen presentaba una estructura dual compuesta por una hoja de reporte institucional (caracterizada por celdas combinadas y formatos agregados) y una hoja etiquetada como BASE en formato de tabla relacional.

Para garantizar la integridad y la idoneidad del procesamiento de datos, la investigación utilizó exclusivamente las hojas BASE. El proceso de consolidación y limpieza estadística resultó en una matriz de datos longitudinal de 13.765 observaciones (filas), la cual abarca un panel de 24 entidades financieras, 25 provincias y 122 cantones, con un registro histórico mensualizado desde enero hasta agosto de 2026.

Tabla 1:
Diccionario de variables

Variable	Descripción
segmento	Tipo de crédito: Consumo, Microcrédito, Productivo, Inmobiliario, Vivienda interés público
fecha / mes	Fecha de corte mensual y su versión AAAA-MM
entidad, provincia, canton	Banco privado y ubicación geográfica de la cartera
region	Calculada: Costa, Sierra, Oriente, Insular (según provincia)
por_vencer	Cartera al día (USD)
no_devenga	Cartera que no devenga intereses (USD)
vencida	Cartera vencida (USD)
saldo_total	Saldo total de la cartera (USD)
improductiva	Calculada: no_devenga + vencida
morosidad_pct	Calculada: improductiva / saldo_total x 100

Proyecto final - Dashboard EDA por profesión



2.2 Métrica clave

Morosidad ponderada = suma de cartera improductiva / suma de saldo total. Se usa la versión ponderada y no el promedio de porcentajes, porque un cantón con USD 2 de saldo no debe pesar igual que uno con USD 2.000 millones.

$$\text{Morosidad Ponderada} = \frac{\sum \text{Cartera Improductiva}}{\sum \text{Saldo Total}}$$

2.3 Estructura del proyecto

```
proyecto/  
|-- app.py      dashboard en Streamlit  
|-- preparar_datos.py      limpieza: data/raw -> data/processed  
|-- requirements.txt librerías necesarias  
|-- README.md descripción, datos y como ejecutar  
|-- notebooks/  
|   +-- EDA_cartera_banca_privada.ipynb      EDA en los 7 pasos  
|-- data/  
|   |-- raw/      4 Excel originales (Superintendencia de Bancos)  
|   +-- processed/      cartera_banca_privada_2026.csv (13.765 filas)  
+-- docs/      este documento en PDF
```

Flujo: los Excel originales (data/raw) se limpian en el notebook o con preparar_datos.py y generan un único CSV (data/processed). Ese CSV lo lee el dashboard, así notebook y tablero usan exactamente los mismos datos.

3 Parte técnica: explicación del código

Esta sección sigue el notebook *EDA_cartera_banca_privada.ipynb* celda por celda. Cada bloque muestra el código y explica qué hace. El nombre de cada paso coincide con los 7 pasos del EDA del curso.

Paso1:

Pregunta ¿Qué segmentos de crédito, regiones geográficas y entidades financieras concentran el mayor riesgo de morosidad dentro del sistema bancario privado ecuatoriano?

Paso 2 ·

Leer la Estructura (carga, limpieza y transformación)

Celda 1: Librerías y rutas

Importa las librerías (pandas y numpy para datos, matplotlib y seaborn para gráficos), define estilo y localiza la carpeta data/raw donde están los Excel originales.

Proyecto final - Dashboard EDA por profesión



Programación y Análisis de Datos (MIACD02P01)

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from pathlib import Path
6 from IPython.display import display

7 sns.set_theme(style="whitegrid") plt.rcParams["figure.dpi"] = 110
8 pd.options.display.float_format = "{:, .2f}".format

# Carpeta del proyecto (funciona ejecutando desde /notebooks o desde la raíz)

9 RAIZ = Path("..")
10 if Path("../data/raw").exists() else Path(".")
11 RAW = RAIZ / "data" / "raw"
12 PROCESADO = RAIZ / "data" / "processed" print("Archivos de entrada:")
13 for f in sorted(RAW.glob("*.xlsx")): print(" -", f.name)
```

Qué hace cada línea

1 y 2 (pandas y numpy): Son las calculadoras y procesadoras de datos. Permiten cargar las bases de datos de la Superintendencia de Bancos y hacer operaciones matemáticas complejas en milisegundos.

3 y 4 (matplotlib y seaborn): Son los diseñadores gráficos. Se encargan de transformar los números de morosidad en gráficos visuales (barras, curvas, etc.).

5 (pathlib): Es el guía de rutas. Ayuda a que el código encuentre los archivos sin importar si estás usando Windows, Mac o Linux.

6 (IPython...display): Es el formateador visual. Sirve para mostrar los resultados en pantalla con tablas limpias y elegantes (formato HTML), en lugar de texto desordenado.

7 (sns.set_theme...): Configura los gráficos con un fondo blanco y cuadrículas grises tenues (whitegrid). El parámetro dpi = 110 le da alta definición a las imágenes para que no se vean borrosas en el informe.

8 (pd.options...): Configura los números financieros. Les añade comas para separar los miles y restringe todo a dos decimales.

9 y 10 (RAIZ = ...): Es un detector inteligente. Si se está ejecutando el código desde la carpeta de notas de programación (/notebooks), retrocede un nivel; si está en la raíz del proyecto, se queda ahí. Esto evita el error de "archivo no encontrado".

11 y 12 (RAW y PROCESADO): Crean las rutas exactas hacia la carpeta donde están los archivos originales descargados (raw) y donde se guardará la gran base de datos consolidada final (processed).

13 for f in sorted(RAW.glob("*.xlsx")): Este bucle escanea la carpeta de datos originales, busca todos los archivos de Excel de la Superintendencia (Consumo, Productivo, Microcrédito y Vivienda) y los imprime en tu pantalla en orden alfabético. Sirve para que el analista confirme que están completos los datos antes de empezar a procesar.

Proyecto final - Dashboard EDA por profesión



Celda 2: Carga y unión de los 4 archivos

Lee cada libro Excel y toma únicamente las hojas que empiezan con «BASE». Usa header=1 porque el encabezado está en la segunda fila, elimina la primera columna vacía, nombra las columnas y rellena hacia abajo el segmento (ffill) porque solo está escrito en la primera fila. Une todo con pd.concat y acorta los nombres de los bancos.

```
14 COLUMNAS = ["segmento", "fecha", "entidad", "provincia", "canton", "por_vencer",  
    "no_devenga", "vencida", "saldo_total"]  
  
15 def entidad_corta(nombre): """Acorta los nombres de banco para que quepan en los  
    graficos.""" n = "  
  
    ".join(nombre.split())  
16 if "CODESARROLLO" in n: return "CODESARROLLO"  
17 if "AMIBANK" in n:  
18     return "AMIBANK (liquidacion)" if "VISIONFUND" in n:  
19     return "VISIONFUND"  
20 return n.replace("BP ", "").replace("BANCO ", "").replace(" S.A.", "")  
21 tablas = []  
22 for archivo in sorted(RAW.glob("*.xlsx")): libro = pd.ExcelFile(archivo)  
23 for hoja in libro.sheet_names:  
24     if hoja.startswith("BASE"): # solo hojas en formato tabla  
25         t = libro.parse(hoja, header=1).iloc[:, 1:] # fila 2 = encabezado; se quita col. A vacia  
26         t.columns = COLUMNAS  
27         t["segmento"] = t["segmento"].ffill() # el segmento solo aparece en la 1a fila  
        tablas.append(t)  
  
28 crudo = pd.concat(tablas, ignore_index=True) crudo["entidad"] =  
29 crudo["entidad"].map(entidad_corta) print("Filas x columnas:", crudo.shape)  
30 crudo.head()
```

Qué hace cada línea

14 (COLUMNAS = ...): Crea una lista con los nombres oficiales que le vas a poner a las columnas de tus tablas para que todas se llamen exactamente igual y no tengan espacios raros.

15 a 20 (def entidad_corta...): Crea una "máquina limpiadora" de nombres. Por ejemplo, si un banco se registra en el Excel como "BANCO PICHINCHA S.A.", esta función le borra el "BANCO" y el "S.A." para dejarlo simplemente como "PICHINCHA". Así tus gráficos no se amontonarán de texto.

21 a 24 (tablas = [] ... if hoja.startswith("BASE")): Crea una lista vacía para guardar los datos. Luego, el código abre uno por uno los 4 archivos Excel (Consumo, Productivo, etc.) y revisa las pestañas internas. Si la pestaña empieza con la palabra "BASE", el código dice: "Esta es la buena" y la procesa; las demás las ignora.

25 (t = libro.parse(...)): Lee la hoja seleccionada. El comando header=1 le dice a Python que los títulos reales de la tabla están en la fila 2 del Excel. El comando .iloc[:, 1:] hace un corte para borrar la columna A, porque venía completamente vacía desde la Superintendencia de Bancos.

Proyecto final - Dashboard EDA por profesión



26 (t.columns = COLUMNAS): Le cambia los nombres originales a las columnas del Excel por los 9 nombres limpios que definiste en la línea 14.

27 (t["segmento"] = ... ffill()): Corrige el gran problema de las celdas combinadas del Excel. Si se quita la combinación de celdas, solo la primera fila se queda con la palabra (ej. "Consumo") y las siguientes quedan vacías (NaN). ffill() copia la palabra hacia abajo de forma automática hasta que encuentra un segmento nuevo. Luego, guarda esa tabla limpia en la lista.

28 y 29 (crudo = pd.concat(...)): Junta (pega hacia abajo) las tablas de los 4 archivos Excel en una sola gran base de datos maestra. Inmediatamente después, le pasa a toda la columna de bancos la "máquina limpiadora" de nombres que programaste antes.

30 (print(...) crudo.head()): Imprime en la pantalla el tamaño final de la tabla (las 13.765 filas y 9 columnas) y muestra las primeras 5 filas para que verifiques visualmente que todo quedó perfecto.

Celda 3: Tipos de datos y dimensiones

Revisa tipos de datos, rango de fechas y cuántas entidades, provincias y cantones hay. Confirma que las fechas se leyeron como fecha y los montos como número.

```
31 crudo.info ()
32 print()

33 print("Periodo:", crudo["fecha"].min().date(), "a", crudo["fecha"].max().date())
34 print("Entidades:", crudo["entidad"].nunique(),
35       "| Provincias:", crudo["provincia"].nunique(),
36       "| Cantones:", crudo["canton"].nunique())
37 print()
38 print(crudo["segmento"].value_counts())
```

Qué hace cada línea

31 crudo.info() Muestra un resumen técnico del conjunto de datos. Te dice cuántas filas y columnas hay, los nombres de cada columna, cuántos datos no están vacíos y el tipo de dato de cada una (texto, números, fechas, etc.).

32 print() Imprime una línea en blanco en la pantalla. Sirve únicamente para separar los resultados visualmente y hacer que la lectura no quede amontonada.

33 print("Periodo:", crudo["fecha"].min().date(), "a", crudo["fecha"].max().date()) Busca la fecha más antigua y la fecha más reciente dentro de la columna "fecha". Luego, las muestra en la pantalla para indicarte el rango de tiempo exacto que cubren los datos (por ejemplo: "Periodo: 2026-01-01 a 2026-12-31").

34 print("Entidades:", crudo["entidad"].nunique(), Cuenta cuántas instituciones o empresas diferentes y únicas hay en la columna "entidad" y muestra ese número. Al dejar la coma al final, el código sabe que la instrucción continúa en las siguientes líneas.

Proyecto final - Dashboard EDA por profesión



Programación y Análisis de Datos (MIACD02P01)

35 "| Provincias:", crudo["provincia"].nunique(), Cuenta cuántas provincias distintas y únicas existen en la columna "provincia". Agrega una barra vertical | en la pantalla para separar visualmente este dato del anterior.

36 "| Cantones:", crudo["canton"].nunique()) Cuenta cuántos cantones únicos hay en la columna "canton", muestra la cantidad y finalmente cierra el paréntesis del print que inició en la línea 34, mostrando los tres conteos en un solo renglón.

37 print() Imprime otra línea en blanco en la pantalla para separar el bloque de conteos geográficos del siguiente resultado.

38 print(crudo["segmento"].value_counts()) Cuenta cuántas veces se repite cada categoría diferente dentro de la columna "segmento". Te muestra una lista ordenada de mayor a menor con el nombre de cada segmento y el total de filas que pertenecen a él.

Celda 4: Diccionario de variables

Documenta cada variable: las 9 originales y las 4 que se calculan más adelante. Sirve para explicar rápidamente los datos ante el docente.

```
39 diccionario = pd.DataFrame({
40 "variable": COLUMNAS + ["improductiva", "morosidad_pct", "mes", "region"],
41 "descripcion": [
42 "Tipo de credito (consumo, microcredito, productivo, inmobiliario, vivienda interes
publico)",
43 "Fecha de corte mensual",
44 "Banco privado", "Provincia", "Canton",
45 "Cartera al día (USD)",
46 "Cartera que no devenga intereses (USD)", "Cartera vencida (USD)",
47 "Saldo total de la cartera (USD)", "Calculada: no_devenga + vencida",
48 "Calculada: improductiva / saldo_total x 100", "Calculada: mes en formato AAAA-MM",
49 "Calculada: Costa, Sierra, Oriente, Insular (asignada por provincia)"],
50 })
51 display(diccionario)
```

Qué hace cada línea

39 diccionario = pd.DataFrame({ Inicia la creación de una tabla estructurada (un DataFrame de la librería Pandas) y la guarda con el nombre "diccionario". Abre una llave { para empezar a definir las columnas de esta nueva tabla.

40 "variable": COLUMNAS + ["improductiva", "morosidad_pct", "mes", "region"], Crea la primera columna llamada "variable". Su contenido será una lista que junta los nombres de las columnas que ya tenías guardadas en la variable COLUMNAS más cuatro campos nuevos que creaste o calcularte después.

41 "descripcion": [Crea la segunda columna llamada "descripcion". Abre un corchete [para comenzar a listar los textos informativos que explicarán qué significa cada una de las variables de la columna

Proyecto final - Dashboard EDA por profesión



anterior.

42 "Tipo de credito (consumo, microcredito, productivo, inmobiliario, vivienda interes publico)", Es el texto explicativo para la primera variable de la lista. Detalla las categorías de crédito que contiene esa columna.

43 "Fecha de corte mensual", Es la descripción para la segunda variable, indicando que se refiere al mes y año al que corresponden los datos de esa fila.

44 "Banco privado", "Provincia", "Canton", Son los textos descriptivos para las siguientes tres variables, aclarando que guardan el nombre de la institución financiera y su ubicación geográfica.

45 "Cartera al dia (USD)", Es la descripción que aclara que esa variable contiene el monto de dinero de los créditos que se están pagando puntualmente.

46 "Cartera que no devenga intereses (USD)", "Cartera vencida (USD)", Son las descripciones para las dos variables que miden los créditos con problemas de pago o atrasos.

47 "Saldo total de la cartera (USD)", "Calculada: no_devenga + vencida", Define la descripción del saldo total y luego explica que la nueva variable "improductiva" se obtiene sumando los dos tipos de cartera vencida o con problemas.

48 "Calculada: improductiva / saldo_total x 100", "Calculada: mes en formato AAAA-MM", Explica la fórmula matemática usada para obtener el porcentaje de morosidad y detalla el formato de texto limpio en el que se guardó la fecha mensual.

49 "Calculada: Costa, Sierra, Oriente, Insular (asignada por provincia)", }) Explica que la variable "region" clasifica geográficamente los datos según su provincia. Cierra la lista de descripciones], la estructura de la tabla } y el comando de Pandas).

50 display(diccionario) Muestra la tabla del diccionario de datos en la pantalla de forma visual, limpia y elegante (con formato de filas y columnas de cuaderno interactivo) para que puedas leer fácilmente qué significa cada variable.

Celda 5: Controles de integridad

Dos controles de integridad. Primero verifica que por vencer + no devenga + vencida cuadre con el saldo total (la diferencia máxima es de centavos). Segundo detecta llaves repetidas: son filas de BP Pichincha en Quito con un monto simbólico de USD 1 junto a la fila principal; se consolidan sumando porque son montos.

```
51 df = crudo.copy()
```

```
# Control 1: las tres partes de la cartera deben sumar el saldo total
```

```
52 df["cuadre"] = df["por_vencer"] + df["no_devenga"] + df["vencida"] - df["saldo_total"]
```

```
53 print("Diferencia maxima en el cuadre (USD):", df["cuadre"].abs().max())
```

Proyecto final - Dashboard EDA por profesión



Programación y Análisis de Datos (MIACD02P01)

```
# Control 2: no debe haber filas repetidas para la misma llave

54 llave = ["segmento", "fecha", "entidad", "provincia", "canton"]
55 print("Filas duplicadas en la llave:", df.duplicated(llave).sum())
56 display(df[df.duplicated(llave, keep=False)].sort_values(llave).head(4))

# Se consolidan sumando (son montos)

57 df = df.groupby(llave, as_index=False)[["por_vencer", "no_devenga", "vencida",
"saldo_total"]].sum()
58 print("Filas despues de consolidar:", len(df))
```

Qué hace cada línea

51 `df = crudo.copy()` Crea una copia exacta del conjunto de datos original "crudo" y la guarda en una nueva variable llamada "df". Esto se hace para poder modificar y limpiar los datos sin alterar el archivo original.

52 `df["cuadre"] = df["por_vencer"] + df["no_devenga"] + df["vencida"] - df["saldo_total"]` Crea una nueva columna llamada "cuadre" para verificar errores. Suma los tres estados de la cartera (al día, con problemas y vencida) y le resta el saldo total registrado; si el resultado no es cero, significa que los datos tienen un error de suma.

53 `print("Diferencia maxima en el cuadro (USD):", df["cuadre"].abs().max())` Busca el error de suma más grande detectado en la línea anterior (sin importar si es positivo o negativo gracias a `.abs()`) y lo muestra en la pantalla para comprobar si los números de los reportes cuadran perfectamente.

54 `llave = ["segmento", "fecha", "entidad", "provincia", "canton"]` Define y guarda una lista de cinco columnas específicas bajo el nombre de "llave". Esta combinación de datos debería ser única por cada fila (por ejemplo: no debería haber dos filas para el mismo banco, el mismo mes, en el mismo cantón y con el mismo tipo de crédito).

55 `print("Filas duplicadas en la llave:", df.duplicated(llave).sum())` Busca e identifica cuántas filas repiten exactamente la misma combinación de datos de la "llave" que definiste antes, y muestra en la pantalla el total de registros repetidos encontrados.

56 `display(df[df.duplicated(llave, keep=False)].sort_values(llave).head(4))` Filtra los datos para encontrar los registros repetidos, los ordena por sus columnas clave para que queden juntos y muestra en la pantalla las primeras 4 filas de ejemplo para que puedas revisar visualmente dónde está la duplicidad.

57 `df = df.groupby(llave, as_index=False)[["por_vencer", "no_devenga", "vencida", "saldo_total"]].sum()` Agrupa todas las filas que compartan la misma "llave" y suma los montos de dinero de sus carteras en un solo registro. Esto elimina los duplicados consolidando los valores financieros de forma correcta en lugar de borrarlos.

58 `print("Filas despues de consolidar:", len(df))` Cuenta cuántas filas en total quedaron en el conjunto de datos "df" después de haber fusionado y sumado los duplicados, y te muestra ese número final en la pantalla.

Proyecto final - Dashboard EDA por profesión



Celda 6: Variables derivadas

Crea las variables de análisis: nombres legibles de segmento, la región (Costa, Sierra, Oriente, Insular) según la provincia, la cartera improductiva, la morosidad por fila y el mes. Supuesto documentado: Santo Domingo de los Tsáchilas se clasifica en Sierra; «Otros» agrupa la zona no delimitada.

```
59 SEGMENTOS = {"CONSUMO": "Consumo", "MICROCREDITO": "Microcrédito", "PRODUCTIVO":  
"Productivo",  
"INMOBILIARIO": "Inmobiliario", "VIVIENDA INTERES PUBLICO": "Vivienda interés público"}  
60 df["segmento"] = df["segmento"].map(SEGMENTOS)  
  
61 REGION = {}  
62 for p in ["EL ORO", "ESMERALDAS", "GUAYAS", "LOS RIOS", "MANABI", "SANTA ELENA"]:  
63     REGION[p] = "Costa"  
64 for p in ["AZUAY", "BOLIVAR", "CAÑAR", "CARCHI", "CHIMBORAZO", "COTOPAXI", "IMBABURA",  
"LOJA", "PICHINCHA", "SANTO DOMINGO DE LOS TSÁCHILAS", "TUNGURAHUA"]:  
65     REGION[p] = "Sierra"  
66 for p in ["MORONA SANTIAGO", "NAPO", "ORELLANA", "PASTAZA", "SUCUMBIOS", "ZAMORA  
CHINCHIPE"]:  
67     REGION[p] = "Oriente"  
68 REGION["GALAPAGOS"] = "Insular"  
  
69 df["region"] = df["provincia"].map(REGION).fillna("Otros") # 'Otros' = zona no delimitada  
70 df["improductiva"] = df["no_devenga"] + df["vencida"]  
71 df["morosidad_pct"] = np.where(df["saldo_total"] > 0, df["improductiva"] /  
df["saldo_total"] * 100, np.nan)  
72 df["mes"] = df["fecha"].dt.strftime("%Y-%m")  
  
73 df.drop(columns="cuadre", errors="ignore", inplace=True)  
74 df.head()
```

Qué hace cada línea

59 SEGMENTOS = {"CONSUMO": "Consumo", "MICROCREDITO": "Microcrédito", "PRODUCTIVO": "Productivo", "INMOBILIARIO": "Inmobiliario", "VIVIENDA INTERES PUBLICO": "Vivienda interés público"} Crea un diccionario de traducción para estandarizar los nombres de los tipos de crédito. Convierte los textos que estaban completamente en mayúsculas y sin tildes a un formato de lectura más limpio y correcto.

60 df["segmento"] = df["segmento"].map(SEGMENTOS) Aplica el diccionario anterior a la columna "segmento". Reemplaza cada nombre viejo por el nuevo formato estandarizado en toda la tabla.

61 a 68 REGION = {} ... REGION["GALAPAGOS"] = "Insular" Crea y llena un diccionario llamado "REGION" mediante varios ciclos. Recorre las provincias de Ecuador una por una y les asigna su región geográfica correspondiente de la siguiente manera: a El Oro, Guayas, etc., les asigna "Costa"; a Azuay, Pichincha, etc., les asigna "Sierra"; a las provincias amazónicas les asigna "Oriente"; y a Galápagos le asigna "Insular".

69 df["region"] = df["provincia"].map(REGION).fillna("Otros") Crea una nueva columna llamada "region" cruzando las provincias de la tabla con el mapa geográfico que acabas de definir. Si encuentra alguna provincia desconocida o zona no delimitada, le asigna automáticamente la etiqueta "Otros"

Proyecto final - Dashboard EDA por profesión



gracias al `.fillna()`.

70 `df["improductiva"] = df["no_devenga"] + df["vencida"]` Calcula y crea la variable "improductiva" sumando el dinero de la cartera que no genera intereses con el de la cartera que ya venció, obteniendo así el monto total de los créditos con problemas de pago.

71 `df["morosidad_pct"] = np.where(df["saldo_total"] > 0, df["improductiva"] / df["saldo_total"] * 100, np.nan)` Calcula el porcentaje de morosidad dividiendo la cartera improductiva para el saldo total y multiplicándolo por 100. Usa una regla de seguridad: si el saldo total es cero, en lugar de dar un error matemático de división para cero, guarda un valor vacío (`np.nan`).

72 `df["mes"] = df["fecha"].dt.strftime("%Y-%m")` Extrae el año y el mes de la fecha original y los convierte a un formato de texto limpio tipo "Año-Mes" (por ejemplo: "2026-10") para poder agrupar o filtrar los reportes mensualmente de forma más sencilla.

73 `df.drop(columns="cuadre", errors="ignore", inplace=True)` Elimina de la tabla la columna temporal "cuadre" que habías creado en los pasos anteriores para el control. El parámetro `errors="ignore"` evita que el código falle si la columna ya no existía, e `inplace=True` hace el borrado directamente en la tabla actual.

74 `df.head()` Muestra en la pantalla las primeras 5 filas de la tabla final ya limpia, estandarizada y con todas las nuevas columnas calculadas para que puedas verificar que el resultado es correcto.

Celda 7: Exportar dataset limpio

Guarda el dataset limpio en `data/processed`. Este CSV es el que lee el dashboard de Streamlit, así notebook y tablero usan exactamente los mismos datos.

```
75 PROCESADO.mkdir(parents=True, exist_ok=True)
76 columnas_salida = ["fecha", "mes", "segmento", "entidad", "provincia", "region", "canton",
77 "por_vencer", "no_devenga", "vencida", "saldo_total", "improductiva", "morosidad_pct"]
78 df[columnas_salida].to_csv(PROCESADO / "cartera_banca_privada_2026.csv", index=False)
79 print("Guardado:", PROCESADO / "cartera_banca_privada_2026.csv", "| filas:", len(df))
```

Qué hace cada línea

75 `PROCESADO.mkdir(parents=True, exist_ok=True)` Crea de forma automática la carpeta donde se guardarán los resultados (la ruta guardada en la variable `PROCESADO`). Si las carpetas superiores no existen, las crea en cadena (`parents=True`), y si la carpeta ya existía, no hace nada ni arroja ningún error (`exist_ok=True`).

76 `columnas_salida = ["fecha", "mes", "segmento", ... "morosidad_pct"]` Define y guarda una lista de texto con los nombres de las columnas exactas que quieres conservar. Sirve para ordenar la estructura final de los datos y dejar fuera cualquier columna temporal o basura que no interese para el reporte.

77 `df[columnas_salida].to_csv(PROCESADO / "cartera_banca_privada_2026.csv", index=False)` Filtra

Proyecto final - Dashboard EDA por profesión



la tabla para quedarse únicamente con las columnas de la lista anterior y la exporta a un archivo de texto separado por comas (.csv) dentro de la carpeta de procesados. El parámetro `index=False` evita que se guarde una columna extra con los números de fila automáticos.

78 `print("Guardado:", PROCESADO / "cartera_banca_privada_2026.csv", "|", len(df))` Imprime un mensaje de confirmación en la pantalla indicando la ruta exacta en donde se creó el archivo definitivo y el número total de filas que se guardaron correctamente en él.

Paso 3 · Univariado

Celda 8: Cobertura de los datos

Cuenta registros por segmento y por mes. Sirve para comprobar que la cobertura es pareja en el tiempo (aprox. 1.700 filas cada mes) y ver qué segmentos tienen más detalle geográfico.

```
79 fig, ax = plt.subplots(1, 2, figsize=(12, 4))
80 df["segmento"].value_counts().sort_values().plot.barh(ax=ax[0], color="#2a6f97")
81 ax[0].set_title("Registros por segmento")
82 df.groupby("mes").size().plot.bar(ax=ax[1], color="#2a6f97", rot=45)
83 ax[1].set_title("Registros por mes")
84 plt.tight_layout()
85 plt.show()
```

Qué hace cada línea

80 `df["segmento"].value_counts().sort_values().plot.barh(ax=ax[0], color="#2a6f97")` Cuenta cuántas filas tiene cada tipo de crédito, ordena los resultados de menor a mayor y dibuja un gráfico de barras horizontales (`barh`) en el primer espacio de la izquierda (`ax[0]`) usando un color azul marino específico.

81 `ax[0].set_title("Registros por segmento")` Le asigna un título descriptivo y claro a la parte superior del primer gráfico (el de la izquierda) para saber qué datos se están visualizando ahí.

82 `df.groupby("mes").size().plot.bar(ax=ax[1], color="#2a6f97", rot=45)` Agrupa los datos según el mes, cuenta la cantidad de registros que tiene cada periodo y dibuja un gráfico de barras verticales (`bar`) en el segundo espacio de la derecha (`ax[1]`), girando las etiquetas de los meses 45 grados (`rot=45`) para que se lean bien.

83 `ax[1].set_title("Registros por mes")` Le coloca el título principal a la parte superior del segundo gráfico (el de la derecha) para identificar el comportamiento del volumen de datos a lo largo del tiempo.

84 `plt.tight_layout()` Ajusta de manera automática los márgenes, textos y espacios entre los dos gráficos. Esto evita que los títulos o las etiquetas de los ejes se amontonen o se corten en los bordes del lienzo.

85 `plt.show()` Procesa todo el diseño final y proyecta la ventana con los dos gráficos listos en la pantalla para que puedas analizarlos visualmente de inmediato.

Proyecto final - Dashboard EDA por profesión



Celda 9: Estadísticos descriptivos

Estadísticos descriptivos (media, mediana, cuartiles, percentil 95) y asimetría. La media del saldo es muy superior a la mediana y la asimetría es alta: pocos cantones concentran casi todo el dinero.

```
86 resumen = df[["saldo_total", "improductiva", "morosidad_pct"]].describe(percentiles=[.25, .5, .75, .95]).T
87 resumen["asimetria"] = df[["saldo_total", "improductiva", "morosidad_pct"]].skew()
88 display(resumen)
```

Qué hace cada línea

86 resumen = df[["saldo_total", "improductiva", "morosidad_pct"]].describe(percentiles=[.25, .5, .75, .95]).T Selecciona las columnas de saldo total, cartera improductiva y porcentaje de morosidad de la tabla, calcula sus estadísticas descriptivas clave (como la media, desviación estándar, mínimos, máximos y los percentiles especificados) y transpone el resultado (.T) para que las variables queden como filas y las estadísticas como columnas.

87 resumen["asimetria"] = df[["saldo_total", "improductiva", "morosidad_pct"]].skew() Calcula el coeficiente de asimetría matemática para las mismas tres variables financieras y lo añade como una nueva columna al resumen estadístico, ayudando a identificar si los datos se concentran en valores bajos o altos.

88 display(resumen) Muestra e imprime la tabla resumen completa en la pantalla con un formato limpio y ordenado para facilitar el análisis inmediato de la distribución de las variables.

Celda 10: Distribución de saldo y morosidad

Histogramas de saldo (en escala logarítmica, porque cubre desde pocos dólares hasta miles de millones) y de morosidad. La morosidad muestra picos en 0% y en 100%: son filas con saldos mínimos, lo que se trata en el paso 5.

```
89 fig, ax = plt.subplots(1, 2, figsize=(12, 4))
90 np.log10(df.loc[df["saldo_total"] > 0, "saldo_total"]).plot.hist(bins=40,
91 ax=ax[0], color="#2a6f97") ax[0].set_title("Saldo por fila (escala log10 de USD)")
92 ax[0].set_xlabel("log10(saldo)") df["morosidad_pct"].plot.hist(bins=50,
93 ax=ax[1], color="#e07a5f")
94 ax[1].set_title("Morosidad por fila (%)")
95 ax[1].set_xlabel("morosidad %")
96 plt.tight_layout()
97 plt.show()
```

Qué hace cada línea

89 fig, ax = plt.subplots(1, 2, figsize=(12, 4)) Crea un lienzo gráfico dividido en una fila con dos columnas independientes (ax[0] y ax[1]) con un tamaño total de 2 pulgadas de ancho por 4 de alto.

Proyecto final - Dashboard EDA por profesión



Programación y Análisis de Datos (MIACD02P01)

90 `np.log10(df.loc[df["saldo_total"] > 0, "saldo_total"]).plot.hist(bins=40, ax=ax[0], color="#2a6f97")` Filtra las filas con saldo mayor a cero, les aplica una transformación matemática de escala logarítmica en base 10 para reducir el sesgo extremo y dibuja un histograma de frecuencias con 40 barras en el primer gráfico de la izquierda usando el color azul corporativo.91 `ax[0].set_title("Saldo por fila (escala log10 de USD)")`

91 Coloca un título descriptivo en la parte superior del primer histograma para identificar que representa la distribución del saldo en escala logarítmica.

92 `ax[0].set_xlabel("log10(saldo)")` Asigna la etiqueta de texto al eje horizontal del primer gráfico indicando que los valores corresponden al logaritmo del saldo.

93 `df["morosidad_pct"].plot.hist(bins=50, ax=ax[1], color="#e07a5f")` Dibuja un segundo histograma de frecuencias con 50 barras en el gráfico de la derecha para evaluar cómo se distribuye el porcentaje de morosidad, utilizando un tono naranja-anaranjado.

94 `ax[1].set_title("Morosidad por fila (%)")` Asigna el título superior correspondiente al gráfico de la derecha para indicar que visualiza los porcentajes de morosidad individuales.

95 `ax[1].set_xlabel("morosidad %")` Coloca la etiqueta de texto en el eje horizontal del segundo gráfico para detallar que mide la morosidad en porcentaje.

96 `plt.tight_layout()` Ajusta automáticamente el espaciado entre los dos histogramas, asegurando que ninguna etiqueta, título o eje se superponga o quede fuera de los márgenes.

97 `plt.show()` Despliega y presenta la figura final con los dos histogramas contruidos en la pantalla.

Celda 11: Peso de cada segmento

Tabla del último mes por segmento: saldo, improductiva, morosidad ponderada y qué porcentaje del saldo y de la improductiva total aporta cada uno. Permite comparar el peso en porcentaje contra el peso en dólares.

```
98 ULT = df["mes"].max() # ultimo mes disponible
99 ago = df[df["mes"] == ULT]

100 seg = ago.groupby("segmento")[["saldo_total", "improductiva"]].sum()
101 seg["morosidad_pct"] = seg["improductiva"] / seg["saldo_total"] * 100
seg["peso_saldo_pct"] = 102 seg["saldo_total"] / seg["saldo_total"].sum() * 100
seg["peso_improductiva_pct"] =
103 seg["improductiva"] / seg["improductiva"].sum() * 100
104 print("Corte:", ULT)
105 display(seg.sort_values("morosidad_pct", ascending=False).round(2))
```

Qué hace cada línea

98 `ULT = df["mes"].max()` Identifica y extrae el valor del último mes cronológico registrado en los

Proyecto final - Dashboard EDA por profesión



datos (en este caso, agosto de 2026) para guardarlo en la variable ULT.

99 ago = df[df["mes"] == ULT] Filtra todo el conjunto de datos para quedarse únicamente con los registros que pertenecen a ese último mes disponible y los guarda en una nueva tabla llamada 'ago'.

100 seg = ago.groupby("segmento")[["saldo_total", "improductiva"]].sum() Agrupa la información de agosto según el tipo de crédito (segmento) y calcula la suma total del saldo de cartera y de la cartera improductiva para cada uno.

101 seg["morosidad_pct"] = seg["improductiva"] / seg["saldo_total"] * 100 Calcula el porcentaje de morosidad agregada para cada segmento de crédito dividiendo la suma acumulada de cartera improductiva entre el saldo total acumulado y multiplicándolo por 100.

102 seg["peso_saldo_pct"] = seg["saldo_total"] / seg["saldo_total"].sum() * 100 Determina el peso porcentual de participación de cada segmento sobre la cartera total del sistema financiero, dividiendo el saldo de cada uno para la suma global.

103 seg["peso_improductiva_pct"] = seg["improductiva"] / seg["improductiva"].sum() * 100 Calcula qué porcentaje de la cartera improductiva de todo el país se concentra en cada segmento específico, dividiendo el valor del segmento para el gran total improductivo.

104 print("Corte:", ULT) Imprime un encabezado de texto en la pantalla que indica explícitamente cuál es el mes de corte de los datos analizados.

105 display(seg.sort_values("morosidad_pct", ascending=False).round(2)) Ordena los segmentos analizados de mayor a menor según su tasa de morosidad, redondea todas las cifras calculadas a dos decimales y muestra la tabla estructurada en pantalla.

Celda 12: Entidades más grandes

Ranking de las 12 entidades con mayor cartera. Muestra que el sistema está concentrado en pocos bancos grandes.

```
106 ent = ago.groupby("entidad")[["saldo_total",  
"improductiva"]].sum().sort_values("saldo_total")  
  
107 fig, ax = plt.subplots(figsize=(9, 6))  
108 (ent["saldo_total"] / 1e9).tail(12).plot.barh(ax=ax, color="#2a6f97")  
109 ax.set_title("Las 12 entidades con mayor cartera (agosto 2026)")  
110 ax.set_xlabel("Miles de millones de USD")  
111 plt.tight_layout()  
112 plt.show()
```

Qué hace cada línea

106 ent = ago.groupby("entidad")[["saldo_total", "improductiva"]].sum().sort_values("saldo_total") Agrupa los datos de agosto por banco o institución financiera, calcula la suma de su saldo e

Proyecto final - Dashboard EDA por profesión



improductiva, y los ordena de menor a mayor en base a su tamaño de cartera total.

107 fig, ax = plt.subplots(figsize=(9, 6)) Inicializa una nueva figura de gráfico con un tamaño de 9 pulgadas de ancho por 6 de alto para albergar el gráfico de barras de las entidades.

108 (ent["saldo_total"] / 1e9).tail(12).plot.barh(ax=ax, color="#2a6f97") Divide el saldo de cartera entre mil millones (1e9) para expresarlo en esa unidad de medida, selecciona las 12 instituciones más grandes del cierre de la lista (.tail(12)) y genera un gráfico de barras horizontales de color azul.

109 ax.set_title("Las 12 entidades con mayor cartera (agosto 2026)") Coloca el título principal descriptivo en la parte superior del gráfico de barras bancario.

110 ax.set_xlabel("Miles de millones de USD") Asigna el rótulo descriptivo al eje horizontal indicando que la unidad de medida representada está expresada en miles de millones de dólares americanos.

111 plt.tight_layout() Ajusta dinámicamente las dimensiones y márgenes del gráfico para asegurar que los nombres de los bancos y las etiquetas numéricas se visualicen completos.

112 plt.show() Presenta formalmente en pantalla el gráfico de barras horizontales generado con el ranking bancario.

Paso 4 · Bivariado

Celda 13: Morosidad por segmento en el tiempo

Relación tiempo-morosidad por segmento. Se agrupa por mes y segmento, se suman saldo e improductiva y se calcula la morosidad ponderada. Microcrédito y consumo se mantienen más altos que productivo e inmobiliario durante todo el periodo.

```
113 mes_seg = df.groupby(["mes", "segmento"])[["saldo_total", "improductiva"]].sum()
114 mes_seg["morosidad_pct"] = mes_seg["improductiva"] / mes_seg["saldo_total"] * 100
115 tabla_mes = mes_seg["morosidad_pct"].unstack()

116 tabla_mes.plot(figsize=(10, 4.5), marker="o")
117 plt.title("Morosidad ponderada por segmento y mes (%)")
118 plt.ylabel("%")
119 plt.xticks(rotation=45)
120 plt.legend(title="", bbox_to_anchor=(1.02, 1), loc="upper left")
121 plt.tight_layout()
122 plt.show()
```

Qué hace cada línea

113 mes_seg = df.groupby(["mes", "segmento"])[["saldo_total", "improductiva"]].sum() Realiza una agrupación multinivel cruzando el mes cronológico y el segmento de crédito para calcular la sumatoria total del saldo y de la cartera improductiva en cada combinación posible.

Proyecto final - Dashboard EDA por profesión



Programación y Análisis de Datos (MIACD02P01)

114 `mes_seg["morosidad_pct"] = mes_seg["improductiva"] / mes_seg["saldo_total"] * 100` Calcula la tasa de morosidad ponderada mensual para cada segmento dividiendo la cartera improductiva sumada para su respectivo saldo total multiplicado por 100.

115 `tabla_mes = mes_seg["morosidad_pct"].unstack()` Reestructura la matriz de datos pivotando los segmentos de crédito para que pasen de estar en filas a convertirse en columnas independientes, facilitando el graficado de series de tiempo.

116 `tabla_mes.plot(figsize=(10, 4.5), marker="o")` Genera un gráfico de líneas a partir de la tabla reestructurada con un tamaño de 10 por 4.5 pulgadas, agregando marcadores circulares ('o') en cada mes para cada segmento representado.

117 `plt.title("Morosidad ponderada por segmento y mes (%))` Asigna el título informativo en la cabecera del gráfico de evolución temporal.

118 `plt.ylabel("%")` Establece la etiqueta del eje vertical indicando que mide valores en términos de porcentaje.

119 `plt.xticks(rotation=45)` Gira las etiquetas de texto de los meses del eje horizontal 45 grados para evitar que se encimen y asegurar una lectura limpia de la línea de tiempo.

120 `plt.legend(title="", bbox_to_anchor=(1.02, 1), loc="upper left")` Configura la leyenda de segmentos quitándole el título y posicionándola por fuera del cuadro del gráfico en la esquina superior derecha para no obstruir las líneas de tendencia.

121 `plt.tight_layout()` Ajusta los bordes externos y la posición de la leyenda externa para que la imagen no se corte al exportarse o mostrarse.

122 `plt.show()` Dibuja y despliega la gráfica lineal temporal final en pantalla.

Celda 14: Segmento x región

Mapa de calor segmento por región. Cruza dos variables categóricas con la morosidad ponderada. Ayuda a ver dónde se concentra el riesgo dentro de cada tipo de crédito. Las celdas vacías significan que no hay cartera de ese segmento en esa región.

```
123 hm = ago.groupby(["segmento", "region"])["saldo_total", "improductiva"].sum()
124 hm["morosidad_pct"] = hm["improductiva"] / hm["saldo_total"] * 100
125 plt.figure(figsize=(9, 4.5))
126 sns.heatmap(hm["morosidad_pct"].unstack(), annot=True, fmt=".1f",
127             cmap="YlOrRd", cbar_kws={"label": "% morosidad"})
127 plt.title("Morosidad ponderada: segmento x región (agosto 2026)")
128 plt.xlabel("")
129 plt.ylabel("")
130 plt.tight_layout()
131 plt.show()
```

Proyecto final - Dashboard EDA por profesión



Qué hace cada línea

123 `hm = ago.groupby(["segmento", "region"])[["saldo_total", "improductiva"]].sum()` Agrupa la información filtrada de agosto cruzando de manera simultánea el tipo de crédito (segmento) y la región geográfica para obtener los totales de saldo e improductiva.

124 `hm["morosidad_pct"] = hm["improductiva"] / hm["saldo_total"] * 100` Determina el indicador de morosidad ponderada para cada intersección de segmento y región dividiendo los totales correspondientes.

125 `plt.figure(figsize=(9, 4.5))` Establece un nuevo espacio para gráfico con dimensiones de 9 pulgadas de ancho por 4.5 de alto especializado para la matriz.

126 `sns.heatmap(hm["morosidad_pct"].unstack(), annot=True, fmt=".1f", cmap="YlOrRd", cbar_kws={"label": "% morosidad"})` Pivota las regiones a columnas y construye un mapa de calor (heatmap). Activa las anotaciones numéricas con un decimal, aplica una paleta de colores de amarillo a rojo (YlOrRd) y etiqueta la barra lateral de intensidad.

127 `plt.title("Morosidad ponderada: segmento x región (agosto 2026)")` Asigna el título aclaratorio en la parte superior del mapa de calor generado.

128 `plt.xlabel("")` Limpia y remueve el título por defecto del eje horizontal para mejorar la estética visual de las regiones.

129 `plt.ylabel("")` Limpia y remueve el título por defecto del eje vertical para mantener despejados los nombres de los segmentos de crédito.

130 `plt.tight_layout()` Corrige la alineación de las etiquetas de texto de filas y columnas para que encajen perfectamente en los cuadrantes de color.

131 `plt.show()` Muestra visualmente el mapa de calor bidimensional en pantalla.

Celda 15: Saldo vs morosidad

Dispersión entre tamaño del saldo y morosidad. Se ve un embudo: con saldos pequeños la morosidad salta entre 0% y 100%, y con saldos grandes se estabiliza. Es evidencia de que los porcentajes de filas pequeñas son ruido, y justifica el paso 5. Se usa correlación de Spearman porque el saldo está muy sesgado.

```
132 plt.figure(figsize=(10, 5))
133 sns.scatterplot(data=ago[ago["saldo_total"] > 0], x="saldo_total", y="morosidad_pct",
134 hue="segmento", alpha=0.5, s=20)
135 plt.xscale("log")
136 plt.title("Saldo vs morosidad por cantón-entidad (agosto 2026)")
137 plt.xlabel("Saldo (USD, escala log)")
138 plt.ylabel("Morosidad %")
139 plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left", title="")
```

Proyecto final - Dashboard EDA por profesión



Programación y Análisis de Datos (MIACD02P01)

```
139 plt.tight_layout()
140 plt.show()
141 print("Correlacion de Spearman saldo-morosidad:", round(ago[["saldo_total",
"morosidad_pct"]].corr(method="spearman").iloc[0, 1], 3))
```

Qué hace cada línea

132 plt.figure(figsize=(10, 5)) Prepara un lienzo de dibujo de 10 pulgadas de ancho por 5 de alto para plasmar el gráfico de dispersión de puntos.

133 sns.scatterplot(data=ago[ago["saldo_total"] > 0], x="saldo_total", y="morosidad_pct", hue="segmento", alpha=0.5, s=20) Crea un gráfico de dispersión analizando los registros con saldo positivo de agosto; ubica el saldo en el eje X, la morosidad en el eje Y, diferencia los puntos por color según su segmento, aplica un 50% de transparencia y define un tamaño de punto pequeño igual a 20.

134 plt.xscale("log") Transforma la escala del eje horizontal a formato logarítmico para distribuir equitativamente los puntos, permitiendo visualizar carteras muy pequeñas y multimillonarias en la misma vista.

135 plt.title("Saldo vs morosidad por cantón-entidad (agosto 2026)") Establece el título descriptivo para el análisis bivariado de dispersión.

136 plt.xlabel("Saldo (USD, escala log)") Asigna el texto indicativo al eje horizontal explicitando el uso de escala logarítmica para los montos.

137 plt.ylabel("Morosidad %") Asigna el texto correspondiente al eje vertical detallando la medición de la tasa de morosidad.

138 plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left", title="") Mueve la leyenda explicativa de los segmentos fuera del área gráfica a la derecha para evitar que se encime con la nube de puntos.

139 plt.tight_layout() Ajusta los componentes y los ejes del gráfico de dispersión logarítmico de forma proporcional.

140 plt.show() Despliega la nube de puntos de dispersión en la pantalla.

141 print("Correlacion de Spearman saldo-morosidad:", round(ago[["saldo_total", morosidad_pct"]].corr(method="spearman").iloc[0, 1], 3)) Calcula la matriz de correlación basada en rangos de Spearman entre saldo total y porcentaje de morosidad, extrae el coeficiente de asociación bivariada, lo redondea a tres decimales y lo imprime en pantalla.

Celda 16: Boxplot por segmento

Diagrama de cajas de la morosidad por segmento, sin filtrar. Los muchos puntos extremos anticipan el tratamiento de atípicos del siguiente paso.

Proyecto final - Dashboard EDA por profesión



Programación y Análisis de Datos (MIACD02P01)

```
142 plt.figure(figsize=(10, 4.5))
143 sns.boxplot(data=df[df["saldo_total"] > 0], x="segmento", y="morosidad_pct",
color="#8ecae6")
144 plt.title("Distribución de la morosidad por fila y segmento (sin filtrar)")
145 plt.xticks(rotation=15)
146 plt.xlabel("")
147 plt.ylabel("Morosidad %")
148 plt.tight_layout()
149 plt.show()
```

Qué hace cada línea

142 plt.figure(figsize=(10, 4.5)) Inicializa un lienzo de visualización de 10 pulgadas de ancho por 4.5 de alto para el diagrama de cajas.

143 sns.boxplot(data=df[df["saldo_total"] > 0], x="segmento", y="morosidad_pct", color="#8ecae6") Construye un diagrama de cajas y bigotes (boxplot) utilizando los registros con saldo de cartera positivo; separa las cajas por cada segmento en el eje X y mide la morosidad en el eje Y, usando un color azul claro.

144 plt.title("Distribución de la morosidad por fila y segmento (sin filtrar)") Establece el título superior del gráfico indicando que representa la distribución completa previa a la limpieza de extremos.

145 plt.xticks(rotation=15) Inclina los nombres de los segmentos de crédito 15 grados en el eje horizontal para optimizar el espacio físico de lectura.

146 plt.xlabel("") Elimina el texto automático del eje de las abscisas para limpiar estéticamente el pie de gráfico.

147 plt.ylabel("Morosidad %") Establece la nomenclatura del eje vertical para recordar la escala porcentual de la morosidad.

148 plt.tight_layout() Corrige la disposición de márgenes para asegurar el encuadre óptimo de las cajas y puntos atípicos.

149 plt.show() Dibuja el diagrama de cajas sin filtrar en la pantalla.

Paso 5 · Faltantes y atípicos

Celda 17: Faltantes

Busca datos faltantes y valores imposibles. No hay nulos ni montos negativos. La morosidad queda como NaN solo cuando el saldo es 0, porque dividir entre cero no está definido. No se imputa nada.

```
150 nulos = df.isna().sum()
151 print("Columnas con nulos:", nulos[nulos > 0].to_dict())
152 print("Filas con saldo 0 (morosidad indefinida):", (df["saldo_total"] == 0).sum())
153 print("Valores negativos en montos:", (df[["por_vencer", "no_devenga", "vencida",
```

Proyecto final - Dashboard EDA por profesión



```
"saldo_total"] < 0).sum().sum())
```

Qué hace cada línea

150 `nulos = df.isna().sum()` Analiza toda la tabla celda por celda identificando cuáles están vacías (nulas) y suma la cantidad total de nulos por cada columna, guardando el resultado en la variable 'nulos'.

151 `print("Columnas con nulos:", nulos[nulos > 0].to_dict())` Filtra e identifica únicamente aquellas columnas que registraron al menos un valor nulo, convierte ese resultado en un formato de diccionario de Python y lo imprime en la pantalla.

152 `print("Filas con saldo 0 (morosidad indefinida):", (df["saldo_total"] == 0).sum())` Evalúa cuántas filas registran un saldo de cartera igual a cero (lo que genera una división indeterminada para calcular el porcentaje) y muestra la suma total encontrada en pantalla.

153 `print("Valores negativos en montos:", (df[["por_vencer", "no_devenga", "vencida", "saldo_total"]] < 0).sum().sum())` Realiza un escaneo de verificación lógica en las cuatro columnas de dinero buscando valores menores a cero (negativos erróneos) y muestra la cantidad total de celdas con anomalías detectadas.

Paso 5 Faltantes y Atípicos

Celda 18: Atípicos con IQR

Detecta atípicos con la regla del rango intercuartílico (IQR): se consideran atípicos los valores fuera de $Q1 - 1,5 \cdot IQR$ y $Q3 + 1,5 \cdot IQR$. Se aplica a saldo y a morosidad.

```
154 def limites_iqr(s):
155     q1, q3 = s.quantile([0.25, 0.75])
156     i = q3 - q1
157     return q1 - 1.5 * i, q3 + 1.5 * i

158 for col in ["saldo_total", "morosidad_pct"]:
159     lo, hi = limites_iqr(df[col].dropna())
160     n = ((df[col] < lo) | (df[col] > hi)).sum()
161     print(f"{col}: límites IQR = [{lo:,.1f}; {hi:,.1f}] -> {n} filas atipicas ({n/len(df):.1%})")
```

Qué hace cada línea

154 `def limites_iqr(s):` Define una función personalizada llamada 'limites_iqr' que recibirá una columna o serie numérica 's' para calcular sus fronteras estadísticas de valores extremos.

155 `q1, q3 = s.quantile([0.25, 0.75])` Dentro de la función, calcula de manera simultánea los valores correspondientes al primer cuartil (percentil 25) y tercer cuartil (percentil 75) de la serie ingresada.

156 `i = q3 - q1` Calcula el Rango Intercuartílico (IQR) restando el valor del primer cuartil al valor del

Proyecto final - Dashboard EDA por profesión



tercer cuartil.

157 `return q1 - 1.5 * i, q3 + 1.5 * i` Devuelve como resultado matemático el límite inferior ($Q1 - 1.5$ veces el IQR) y el límite superior ($Q3 + 1.5$ veces el IQR) que determinan el rango de aceptación estándar.

158 `for col in ["saldo_total", "morosidad_pct"]:` Inicia un bucle que se ejecutará dos veces para evaluar de forma consecutiva la columna de saldo total y luego la columna de porcentaje de morosidad.

159 `lo, hi = limites_iqr(df[col].dropna())` Llama a la función de cuartiles omitiendo valores vacíos para obtener las fronteras numéricas inferior ('lo') y superior ('hi') de la columna en turno.

160 `n = ((df[col] < lo) | (df[col] > hi)).sum()` Cuenta cuántas filas dentro de la columna evaluada se sitúan por debajo del límite inferior o por encima del límite superior, considerándose técnicamente como valores atípicos.

161 `print(f'{col}: limites IQR = [{lo:.1f}; {hi:.1f}] -> {n} filas atípicas ({n/len(df):.1%})'` Muestra un reporte formateado que detalla el nombre de la variable, los valores exactos de sus límites IQR, la cantidad absoluta de filas fuera de rango y su peso porcentual respecto al total de observaciones.

Celda 19: Filas de saldo mínimo

Cuantifica el problema: alrededor del 16% de las filas tiene saldo menor a USD 100 mil, pero pesan una fracción mínima del saldo total. Casi todas las filas con morosidad de 100% son de saldos diminutos. Son atípicos por tamaño, no por riesgo real.

```
162 MIN_SALDO = 100_000 # supuesto: menos de USD 100 mil no permite medir una tasa con
sentido
163 pequenas = df[df["saldo_total"] < MIN_SALDO]

164 print(f"Filas con saldo < USD {MIN_SALDO:,}: {len(pequenas):,}
({len(pequenas)/len(df):.1%} de las filas)")
165 print(f"...pero solo {pequenas['saldo_total'].sum()/df['saldo_total'].sum():.3%} del
saldo total")
166 print(f"...y {pequenas['improductiva'].sum()/df['improductiva'].sum():.3%} de la
improductiva total")
167 print("Filas con morosidad >= 99.9%:", (df["morosidad_pct"] >= 99.9).sum())
168 print(" de ellas con saldo < 100 mil:", ((df["morosidad_pct"] >= 99.9) &
(df["saldo_total"] < MIN_SALDO)).sum())
```

Qué hace cada línea

162 `MIN_SALDO = 100_000` Establece una variable de frontera de negocio fijada en 100,000 dólares para clasificar y separar los registros con saldos financieramente muy pequeños o insignificantes.

163 `pequenas = df[df["saldo_total"] < MIN_SALDO]` Crea una subtabla llamada 'pequenas' filtrando todas las filas de la base de datos cuyo saldo total reportado sea estrictamente menor a la constante de 100,000 dólares.

Proyecto final - Dashboard EDA por profesión



164 print(f'Filas con saldo < USD {MIN_SALDO:,}: {len(pequenas):,} ({len(pequenas)/len(df):.1%} de las filas)") Muestra en la pantalla el conteo absoluto de filas con saldos menores a 100,000 dólares y calcula qué porcentaje del volumen total de la base representan.

165 print(f' ...pero solo {pequenas['saldo_total'].sum()/df['saldo_total'].sum():.3%} del saldo total") Calcula e imprime la participación financiera real de estas filas pequeñas dividiendo su saldo sumado entre la gran suma nacional, demostrando su peso insignificante.

166 print(f' ...and {pequenas['improductiva'].sum()/df['improductiva'].sum():.3%} de la improductiva total") Calcula y muestra la proporción de cartera improductiva de estas filas respecto al total país para evaluar el impacto real de riesgo de estas cuentas menores.

167 print("Filas con morosidad >= 99.9%:", (df["morosidad_pct"] >= 99.9).sum()) Busca y cuenta de manera absoluta cuántas observaciones en la tabla registran tasas de morosidad prácticamente totales (iguales o superiores al 99.9%).

168 print(" de ellas con saldo < 100 mil:", ((df["morosidad_pct"] >= 99.9) & (df["saldo_total"] < MIN_SALDO)).sum()) Cruza ambas condiciones lógicas para contar cuántas de las filas con morosidad extrema corresponden en realidad a carteras con saldos insignificantes menores a 100,000 dólares.

Celda 20: Antes y después del filtro

Aplica el tratamiento y compara antes y después con boxplots. Tras filtrar por saldo mínimo, la dispersión extrema desaparece y quedan tasas comparables.

```
169 analisis = df[df["saldo_total"] >= MIN_SALDO].copy() # para rankings a nivel canton-entidad
170 fig, ax = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
171 sns.boxplot(data=df[df["saldo_total"] > 0], x="segmento", y="morosidad_pct",
172 color="#e07a5f", ax=ax[0]) ax[0].set_title("Antes (todas las filas)")
173 sns.boxplot(data=analisis, x="segmento", y="morosidad_pct", color="#8ecae6", ax=ax[1])
174 ax[1].set_title("Despues (saldo >= USD 100 mil)")
175 for a_ in ax:
176     a_.set_xlabel("")
177     a_.tick_params(axis="x", rotation=20)
178 plt.tight_layout()
179 plt.show()
```

Qué hace cada línea

169 analisis = df[df["saldo_total"] >= MIN_SALDO].copy() Crea un conjunto de datos depurado llamado 'analisis' conservando solo los registros con saldos representativos (mayores o iguales a 100,000 dólares) para aislar el ruido estadístico.

170 fig, ax = plt.subplots(1, 2, figsize=(12, 4), sharey=True) Crea una figura compuesta por dos secciones gráficas paralelas que compartirán de forma idéntica la escala del eje vertical ('sharey=True'), midiendo 12x4 pulgadas.

Proyecto final - Dashboard EDA por profesión



171 `sns.boxplot(data=df[df["saldo_total"] > 0], x="segmento", y="morosidad_pct", color="#e07a5f", ax=ax[0])` Dibuja el diagrama de cajas original con todas las filas con saldo en la sección de la izquierda (`ax[0]`), pintándolo con un tono terracota para el grupo 'Antes'.

172 `ax[0].set_title("Antes (todas las filas)")` Asigna el título que identifica el estado inicial sin filtrar sobre el primer gráfico.

173 `sns.boxplot(data= analisis, x="segmento", y="morosidad_pct", color="#8ecae6", ax=ax[1])` Dibuja el nuevo diagrama de cajas depurado en la sección de la derecha (`ax[1]`) utilizando la tabla de carteras significativas, coloreado de azul claro para representar el 'Después'.

174 `ax[1].set_title("Despues (saldo >= USD 100 mil)")` Asigna el título correspondiente al estado depurado sobre el segundo gráfico.

175 `for a_ in ax:` Inicia un bucle iterativo corto para aplicar modificaciones de formato de manera uniforme a ambas secciones gráficas a la vez.

176 `a_.set_xlabel("")` Elimina las etiquetas del eje horizontal dentro del bucle para limpiar el diseño visual de los ejes.

177 `a_.tick_params(axis="x", rotation=20)` Aplica una rotación de 20 grados a los nombres de los segmentos en el eje X de ambos gráficos para asegurar su correcta lectura.

178 `plt.tight_layout()` Sincroniza y recalcula los espacios y márgenes totales del lienzo compartido para garantizar una visualización simétrica.

179 `plt.show()` Despliega la figura final comparativa de diagramas de caja en pantalla.

Paso 6 · Segmentación

Celda 21: Segmento: evolución y crecimiento

Segmentación por tipo de crédito: morosidad de enero contra agosto, variación en puntos porcentuales y crecimiento de la cartera en el periodo. Vivienda de interés público tiene un crecimiento enorme pero parte de una base muy pequeña, por lo que hay que interpretarlo con cuidado.

```
180 saldo_mes = df.groupby(["mes", "segmento"])["saldo_total"].sum().unstack()
181 cambio = pd.DataFrame({
    "morosidad_ene": tabla_mes.iloc[0],
    "morosidad_ago": tabla_mes.iloc[-1],
})
182 cambio["var_pp"] = cambio["morosidad_ago"] - cambio["morosidad_ene"]
183 cambio["crecimiento_cartera_pct"] = (saldo_mes.iloc[-1] / saldo_mes.iloc[0] - 1) * 100
184 display(cambio.sort_values("morosidad_ago", ascending=False).round(2))
```



Qué hace cada línea

180 `saldo_mes = df.groupby(["mes", "segmento"])["saldo_total"].sum().unstack()` Agrupa el saldo total cruzando mes y segmento de crédito, y pivota la estructura (`.unstack()`) para crear una tabla histórica donde los meses son filas y los segmentos columnas.

181 `cambio = pd.DataFrame({ "morosidad_ene": tabla_mes.iloc[0], "morosidad_ago": tabla_mes.iloc[-1], })` Construye una nueva tabla comparativa ('cambio') tomando los valores de la tasa de morosidad de la primera fila (`.iloc[0]`) correspondiente a enero) y de la última fila (`.iloc[-1]`) correspondiente a agosto).

182 `cambio["var_pp"] = cambio["morosidad_ago"] - cambio["morosidad_ene"]` Calcula la variación neta absoluta en puntos porcentuales de la morosidad restando la tasa de enero a la tasa registrada en agosto.

183 `cambio["crecimiento_cartera_pct"] = (saldo_mes.iloc[-1] / saldo_mes.iloc[0] - 1) * 100` Calcula la tasa de crecimiento porcentual del saldo total de la cartera de cada segmento financiero, comparando el monto del cierre de agosto frente a la base inicial de enero.

184 `display(cambio.sort_values("morosidad_ago", ascending=False).round(2))` Ordena los resultados consolidados de mayor a menor en base a la morosidad final de agosto, redondea los indicadores a dos decimales y los proyecta tabulados en la pantalla.

Celda 22: Región: tasa y dólares

Segmentación por región, mostrando lado a lado el porcentaje y los dólares. Sirve para contrastar la hipótesis H2 (diferencias regionales). Se debe leer con ambos gráficos: una región puede tener tasa alta pero pocos dólares en juego.

```
185 reg = ago.groupby("region")["saldo_total", "improductiva"].sum()
186 reg["morosidad_pct"] = reg["improductiva"] / reg["saldo_total"] * 100
187 reg = reg.sort_values("morosidad_pct", ascending=False)

188 fig, ax = plt.subplots(1, 2, figsize=(12, 4))
189 reg["morosidad_pct"].plot.bar(ax=ax[0], color="#e07a5f", rot=0)
190 ax[0].set_title("Morosidad ponderada por región (%)")
191 (reg["improductiva"] / 1e6).plot.bar(ax=ax[1], color="#2a6f97", rot=0)
192 ax[1].set_title("Cartera improductiva por región (millones USD)")
193 for a_ in ax
194     a_.set_xlabel("")
195 plt.tight_layout()
196 plt.show()
197 display(reg.round(2))
```

Qué hace cada línea

185 `reg = ago.groupby("region")["saldo_total", "improductiva"].sum()` Agrupa la información total agregada de agosto por cada región geográfica y computa la suma total acumulada del saldo y de la cartera improductiva.

Proyecto final - Dashboard EDA por profesión



186 `reg["morosidad_pct"] = reg["improductiva"] / reg["saldo_total"] * 100` Determina el índice de morosidad ponderada regional dividiendo la sumatoria improductiva respectiva para la sumatoria de saldo total multiplicada por 100.

187 `reg = reg.sort_values("morosidad_pct", ascending=False)` Reordena la tabla de las regiones de forma descendente, posicionando arriba la región con la tasa de morosidad más severa.

188 `fig, ax = plt.subplots(1, 2, figsize=(12, 4))` Inicializa un lienzo gráfico compuesto por una fila y dos columnas de visualización independientes con dimensiones estandarizadas de 12 por 4 pulgadas.

189 `reg["morosidad_pct"].plot.bar(ax=ax[0], color="#e07a5f", rot=0)` Dibuja un gráfico de barras verticales de color terracota en la sección izquierda (`ax[0]`) mostrando los porcentajes de morosidad regionales, manteniendo las etiquetas rectas (`rot=0`).

190 `ax[0].set_title("Morosidad ponderada por región (%"))` Asigna el título descriptivo superior al gráfico porcentual de la izquierda.

191 `(reg["improductiva"] / 1e6).plot.bar(ax=ax[1], color="#2a6f97", rot=0)` Divide el dinero improductivo entre un millón (1e6) para convertirlo a esa escala, y traza un gráfico de barras verticales azules en la sección derecha (`ax[1]`) con orientación recta.

192 `ax[1].set_title("Cartera improductiva por región (millones USD))` Asigna el título explicativo correspondiente al gráfico de montos absolutos en millones de la derecha.

193 `for a_ in ax:` Inicia un recorrido iterativo automático aplicado a los dos gráficos del lienzo regional.

194 `a_.set_xlabel("")` Borra el rótulo del eje horizontal en ambos gráficos para simplificar y dar mayor espacio al nombre de las regiones.

195 `plt.tight_layout()` Ajusta los límites, márgenes y alineación de barras de ambos gráficos regionales de forma limpia.

196 `plt.show()` Despliega la composición de barras de tasas y millones en la pantalla.

197 `display(reg.round(2))` Muestra en la pantalla la matriz de datos numéricos regional con formato redondeado a dos cifras decimales.

Celda 23: Provincias más morosas

Ranking de las 10 provincias con más morosidad ponderada, excluyendo las de cartera muy pequeña para no comparar tasas sobre montos mínimos. La línea punteada marca el promedio del sistema.

```
198 PROV_MIN = 50_000_000 # solo provincias con al menos USD 50 millones de cartera
199 prov = ago.groupby("provincia")[["saldo_total", "improductiva"]].sum()
200 prov["morosidad_pct"] = prov["improductiva"] / prov["saldo_total"] * 100
201 top_prov = prov[prov["saldo_total"] >= PROV_MIN].sort_values("morosidad_pct",
```

Proyecto final - Dashboard EDA por profesión



Programación y Análisis de Datos (MIACD02P01)

```
ascending=False).head(10)

202 fig, ax = plt.subplots(figsize=(9, 5))
203 top_prov["morosidad_pct"].sort_values().plot.barh(ax=ax, color="#e07a5f")
204 ax.axvline(ago["improductiva"].sum() / ago["saldo_total"].sum() * 100, color="k", ls="--",
205            label="Promedio sistema")
206 ax.set_title("10 provincias con mayor morosidad (cartera >= USD 50 millones)")
207 ax.set_xlabel("Morosidad %")
208 ax.legend()
209 plt.tight_layout()
210 plt.show()
```

Qué hace cada línea

198 PROV_MIN = 50_000_000 Define un umbral financiero de 50 millones de dólares como requisito de tamaño de cartera para incluir provincias en la evaluación de rankings de mora.

199 prov = ago.groupby("provincia")[["saldo_total", "improductiva"]].sum() Agrupa las observaciones de agosto según su provincia geográfica y calcula la suma total del saldo de cartera y el dinero improductivo.

200 prov["morosidad_pct"] = prov["improductiva"] / prov["saldo_total"] * 100 Calcula el índice de morosidad ponderada provincial dividiendo el monto improductivo acumulado para el saldo de cartera total de la provincia por 100.

201 top_prov = prov[prov["saldo_total"] >= PROV_MIN].sort_values("morosidad_pct", ascending=False).head(10) Filtra para conservar solo las provincias que superen los 50 millones en cartera, las ordena de mayor a menor morosidad y extrae las 10 provincias líderes en mora.

202 fig, ax = plt.subplots(figsize=(9, 5)) Prepara un lienzo para gráfico individual con un tamaño de 9 pulgadas de ancho por 5 de alto.

203 top_prov["morosidad_pct"].sort_values().plot.barh(ax=ax, color="#e07a5f") Ordena las 10 provincias de menor a mayor y genera un gráfico de barras horizontales de color naranja-terracota en el eje dispuesto.

204 ax.axvline(ago["improductiva"].sum() / ago["saldo_total"].sum() * 100, color="k", ls="--", label="Promedio sistema") Traza una línea de referencia vertical discontinua ('--') de color negro ('k') que marca matemáticamente la tasa de morosidad promedio de todo el sistema bancario nacional.

205 ax.set_title("10 provincias con mayor morosidad (cartera >= USD 50 millones)") Asigna el título descriptivo superior al gráfico de barras de las provincias morosas.

206 ax.set_xlabel("Morosidad %") Coloca la etiqueta al eje horizontal para denotar la medición en escala porcentual.

207 ax.legend() Activa e inserta el cuadro de leyenda para identificar el significado de la línea de

Proyecto final - Dashboard EDA por profesión

promedio nacional.

208 plt.tight_layout() Ajusta los márgenes exteriores para evitar el truncamiento de los nombres largos de las provincias de Ecuador.

209 plt.show() Muestra en la pantalla la gráfica del top 10 de provincias críticas.

Paso 7 Hallazgos

Celda 24: Matriz de riesgo por entidad

Matriz de riesgo por entidad. Cada burbuja es un banco: eje X = tamaño de cartera, eje Y = morosidad, tamaño = dólares improductivos. Los bancos se clasifican en pequeño, mediano y grande por terciles de cartera. La línea punteada es la morosidad promedio del sistema.

```
210 ENT_MIN = 100_000_000 # entidades con al menos USD 100 millones
211 en = ago.groupby("entidad")[["saldo_total", "improductiva"]].sum()
212 en["morosidad_pct"] = en["improductiva"] / en["saldo_total"] * 100
213 en = en[en["saldo_total"] >= ENT_MIN].copy()
214 en["tamano"] = pd.qcut(en["saldo_total"], 3, labels=["Pequeña", "Mediana", "Grande"])
215 prom = ago["improductiva"].sum() / ago["saldo_total"].sum() * 100

216 fig, ax = plt.subplots(figsize=(10, 6))
217 sns.scatterplot(data=en, x="saldo_total", y="morosidad_pct", size="improductiva",
218 hue="tamano", sizes=(60, 900), alpha=0.6, ax=ax)
219 ax.axhline(prom, color="k", ls="--", lw=1)
220 ax.set_xscale("log")
221 ax.set_title("Matriz de riesgo por entidad (tamaño de la burbuja = cartera improductiva)")
222 ax.set_xlabel("Cartera total (USD, escala log)")
223 ax.set_ylabel("Morosidad %")
224 desplazamientos = [(8, 8), (10, 30), (-90, -28), (12, -22), (8, 8), (8, 8)]# evita que se
225 superpongan
226 top6 = en.sort_values("improductiva", ascending=False).head(6)
227 for (nombre, fila), desp in zip(top6.iterrows(), desplazamientos):
228     ax.annotate(nombre, (fila["saldo_total"], fila["morosidad_pct"]), fontsize=8,
229                 xytext=desp, textcoords="offset points",
230                 arrowprops=dict(arrowstyle="-", lw=0.6, color="gray"))
231 ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)
232 plt.tight_layout()
233 plt.show()
```

Qué hace cada línea

210 ENT_MIN = 100_000_000 Fija un límite de 100 millones de dólares de saldo total para filtrar los bancos e incluir en la matriz solo instituciones bancarias con una escala de operación intermedia o alta.

211 en = ago.groupby("entidad")[["saldo_total", "improductiva"]].sum() Agrupa el archivo consolidado de agosto por cada banco o entidad financiera y suma sus saldos y montos improductivos totales.

212 en["morosidad_pct"] = en["improductiva"] / en["saldo_total"] * 100 Calcula el porcentaje de morosidad ponderada por institución financiera dividiendo el total improductivo para el saldo por 100.

213 en = en[en["saldo_total"] >= ENT_MIN].copy() Filtra la tabla de bancos reteniendo únicamente

Proyecto final - Dashboard EDA por profesión



aquellos que cumplen con el saldo mínimo establecido de 100 millones de dólares.

214 `en["tamano"] = pd.qcut(en["saldo_total"], 3, labels=["Pequeña", "Mediana", "Grande"])` Divide matemáticamente a los bancos filtrados en tres grupos iguales (terciles) según su saldo y les asigna las etiquetas ordinales 'Pequeña', 'Mediana' y 'Grande'.

215 `prom = ago["improductiva"].sum() / ago["saldo_total"].sum() * 100` Calcula el indicador de morosidad global ponderada del sistema para utilizarlo como referencia estandarizada en el plano.

216 `fig, ax = plt.subplots(figsize=(10, 6))` Inicializa un área gráfica amplia de 10 por 6 pulgadas para construir el plano tridimensional de burbujas.

217 `sns.scatterplot(data=en, x="saldo_total", y="morosidad_pct", size="improductiva", hue="tamano", sizes=(60, 900), alpha=0.6, ax=ax)` Genera un gráfico de dispersión avanzado: ubica el saldo en X, la morosidad en Y; el tamaño de la burbuja varía según el dinero improductivo y el color según el grupo de tamaño, regulando diámetros y aplicando transparencia.

218 `ax.axhline(prom, color="k", ls="--", lw=1)` Dibuja una línea horizontal de referencia punteada de color negro y ancho de línea igual a 1 que señala la tasa de morosidad promedio nacional.

219 `ax.set_xscale("log")` Configura el eje de las abscisas en escala logarítmica para poder mapear equitativamente bancos medianos y colosos corporativos sin amontonar los círculos.

220 `ax.set_title("Matriz de riesgo por entidad (tamaño de la burbuja = cartera improductiva)")` Asigna el título institucional explicativo en la parte alta de la matriz de burbujas.

221 `ax.set_xlabel("Cartera total (USD, escala log)")` Asigna la etiqueta indicativa al eje horizontal detallando los montos en escala logarítmica.

222 `ax.set_ylabel("Morosidad %")` Establece el rótulo del eje vertical identificando la escala de porcentaje de morosidad.

223 `desplazamientos = [(8, 8), (10, 30), (-90, -28), (12, -22), (8, 8), (8, 8)]` Define una lista fija de coordenadas de desfase en puntos para colocar el texto de los nombres de los bancos y evitar superposiciones visuales entre etiquetas.

224 `top6 = en.sort_values("improductiva", ascending=False).head(6)` Filtra la tabla de bancos para identificar y extraer las 6 instituciones con mayor volumen absoluto de dinero en cartera improductiva para etiquetarlas de forma obligatoria.

225 `for (nombre, fila), desp in zip(top6.iterrows(), desplazamientos):` Inicia un ciclo combinado que recorre en simultáneo las filas de las 6 entidades críticas y su respectiva coordenada de desplazamiento de texto.

226 `ax.annotate(nombre, (fila["saldo_total"], fila["morosidad_pct"]), fontsize=8, xytext=desp,`

Proyecto final - Dashboard EDA por profesión



`textcoords="offset points", arrowprops=dict(arrowstyle="-", lw=0.6, color="gray"))` Aplica una anotación de texto con el nombre del banco apuntando a su burbuja, usando una flecha fina gris, posicionando el texto según el desplazamiento para máxima claridad.

227 `ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)` Ubica la leyenda de colores y tamaños fuera de la matriz a la derecha usando un tamaño de letra compacto de 8 puntos.

228 `plt.tight_layout()` Optimiza todos los elementos espaciales de la matriz para que no se corten textos ni leyendas.

229 `plt.show()` Despliega y visualiza la matriz tridimensional de burbujas bancarias en pantalla.

Celda 25: Concentración de la improductiva

Análisis de concentración (tipo Pareto): ordena las entidades por dólares improductivos y calcula el porcentaje acumulado. Contrasta la hipótesis H3.

```
230 pareto = (ago.groupby("entidad")["improductiva"].sum().sort_values(ascending=False))
231 pareto_df = pd.DataFrame({"improductiva_musd": pareto / 1e6, "acumulado_pct":
232 pareto.cumsum() / pareto.sum() * 100}) display(pareto_df.head(8).round(1))
233 print(f"Las 5 primeras entidades concentran {pareto_df['acumulado_pct'].iloc[4]:.1f}% de
la cartera improductiva")
```

Qué hace cada línea

230 `pareto = (ago.groupby("entidad")["improductiva"].sum().sort_values(ascending=False))` Agrupa la cartera improductiva de agosto por banco, calcula la suma de dinero con problemas y la ordena de mayor a menor para iniciar el análisis de concentración estilo Pareto.

231 `pareto_df = pd.DataFrame({"improductiva_musd": pareto / 1e6, "acumulado_pct": pareto.cumsum() / pareto.sum() * 100})` Crea una nueva tabla (`pareto_df`) con dos columnas: los millones de dólares improductivos (dividido para un millón) y el porcentaje acumulado obtenido mediante la suma acumulada dividida para el total global por 100.

232 `display(pareto_df.head(8).round(1))` Muestra en la pantalla las primeras 8 instituciones que concentran el mayor riesgo del país con cifras redondeadas a un decimal.

233 `print(f"Las 5 primeras entidades concentran {pareto_df['acumulado_pct'].iloc[4]:.1f}% de la cartera improductiva")` Extrae el valor del porcentaje acumulado de la quinta fila (`.iloc[4]`), e imprime una conclusión que cuantifica la concentración del riesgo de impago en el top 5 bancario.

Celda 26: Cantones con más improductiva

Los 8 cantones con más dólares improductivos (usando solo filas con saldo mínimo, según el paso 5). Muestra dónde está el problema en términos absolutos.

```
234 cant = analisis[analisis["mes"] == ULT].groupby(["provincia", "canton"])[["saldo_total",
```

Proyecto final - Dashboard EDA por profesión



Programación y Análisis de Datos (MIACD02P01)

```
"improductiva"].sum() cant["morosidad_pct"] =  
235 cant["improductiva"] / cant["saldo_total"] * 100  
236 display(cant.sort_values("improductiva", ascending=False).head(8).round(1))
```

Qué hace cada línea

234 `cant = analisis[analisis["mes"] == ULT].groupby(["provincia", "canton"])[["saldo_total", "improductiva"]].sum()` Filtra la tabla depurada de saldos significativos para el último mes, agrupa por provincia y cantón simultáneamente y calcula los totales de saldo e improductiva.

235 `cant["morosidad_pct"] = cant["improductiva"] / cant["saldo_total"] * 100` Calcula el porcentaje de morosidad ponderada a nivel cantonal dividiendo la cartera improductiva local sumada para su saldo de cartera local por 100.

236 `display(cant.sort_values("improductiva", ascending=False).head(8).round(1))` Ordena los cantones de mayor a menor en base al volumen total de dinero con problemas, muestra en pantalla los 8 cantones más críticos y redondea a un decimal.

Celda 27: Hallazgos calculados

Calcula y muestra las conclusiones numéricas a partir de las tablas construidas en los pasos anteriores. Así el texto nunca queda desactualizado respecto a los datos.

```
237 tot = ago["improductiva"].sum() / ago["saldo_total"].sum() * 100  
238 ene = df[df["mes"] == df["mes"].min()]  
239 tot_ene = ene["improductiva"].sum() / ene["saldo_total"].sum() * 100  
  
240 print(f"1. Morosidad total banca privada: {tot:.2f}% en {ULT} (enero: {tot_ene:.2f}%).")  
241 top_seg = seg.sort_values("morosidad_pct", ascending=False)  
242 print(f"2. Segmento mas moroso: {top_seg.index[0]}  
{top_seg['morosidad_pct'].iloc[0]:.2f}%; " f"el menos moroso: {top_seg.index[-1]}  
{top_seg['morosidad_pct'].iloc[-1]:.2f}%).")  
243 mayor_usd = seg["improductiva"].idxmax()  
244 print(f"3. En dolares, {mayor_usd} concentra {seg.loc[mayor_usd,  
'peso_improductiva_pct']:.1f}% de la improductiva " f"(USD {seg.loc[mayor_usd,  
'improductiva']/1e6:,.0f} millones).")  
245 print(f"4. Region con mayor tasa: {reg.index[0]} ({reg['morosidad_pct'].iloc[0]:.2f}%), " f"pero la region con mas dolares improductivos es {reg['improductiva'].idxmax()} " f"(USD {reg['improductiva'].max()/1e6:,.0f} millones).")  
246 print(f"5. Provincia mas morosa (cartera >= USD 50 millones): {top_prov.index[0]}  
{top_prov['morosidad_pct'].iloc[0]:.2f}%).")  
247 print(f"6. Las 5 entidades con mas improductiva concentran  
{pareto_df['acumulado_pct'].iloc[4]:.1f}% del total; " f"la primera es {pareto_df.index[0]}).")  
248 grandes = seg.index[seg["peso_saldo_pct"] >= 1] # se ignoran segmentos con menos de 1%  
del saldo  
249 sube = cambio.loc[grandes, "var_pp"].sort_values()  
250 print(f"7. Variacion ene-ago (segmentos con >= 1% del saldo): mayor baja en  
{sube.index[0]} ({sube.iloc[0]:+.2f} pp); " f"mayor alza en {sube.index[-1]} ({sube.iloc[-1]:+.2f} pp).")
```

Proyecto final - Dashboard EDA por profesión



Qué hace cada línea

237 `tot = ago["improductiva"].sum() / ago["saldo_total"].sum() * 100` Calcula el indicador final de morosidad ponderada agregada del sistema financiero para el cierre de agosto de 2026.

238 `ene = df[df["mes"] == df["mes"].min()]` Busca y filtra todos los registros pertenecientes al primer mes cronológico de la base de datos (enero 2026) y los almacena en la tabla temporal 'ene'.

239 `tot_ene = ene["improductiva"].sum() / ene["saldo_total"].sum() * 100` Calcula el indicador de morosidad ponderada inicial del sistema bancario privado correspondiente al mes de enero.

240 `print(f"1. Morosidad total banca privada: {tot:.2f}% en {ULT} (enero: {tot_ene:.2f}%).")` Imprime en pantalla la primera conclusión formal automatizada comparando la tasa agregada de cierre contra el inicio de año con dos decimales.

241 `top_seg = seg.sort_values("morosidad_pct", ascending=False)` Toma el resumen de segmentos y lo reordena de forma descendente colocando arriba el portafolio de crédito con mayor tasa de morosidad.

242 `print(f"2. Segmento mas moroso: {top_seg.index[0]} ({top_seg['morosidad_pct'].iloc[0]:.2f}%); el menos moroso: {top_seg.index[-1]} ({top_seg['morosidad_pct'].iloc[-1]:.2f}%).")` Extrae e imprime los nombres y las tasas exactas del segmento de crédito en el primer lugar (más moroso) y del último lugar (más sano).

243 `mayor_usd = seg["improductiva"].idxmax()` Identifica mediante `idxmax()` el nombre del segmento específico que acumula la mayor cantidad absoluta de dinero en dólares improductivos.

244 `print(f"3. En dolares, {mayor_usd} concentra {seg.loc[mayor_usd, 'peso_improductiva_pct']:.1f}% de la improductiva (USD {seg.loc[mayor_usd, 'improductiva']/1e6:,.0f} millones).")` Extrae el peso porcentual y calcula los millones de dólares con problemas de ese segmento líder en riesgo absoluto, imprimiendo el resultado con formato financiero.

245 `print(f"4. Region con mayor tasa: {reg.index[0]} ({reg['morosidad_pct'].iloc[0]:.2f}%), pero la region con mas dolares improductivos es {reg['improductiva'].idxmax()} (USD {reg['improductiva'].max()/1e6:,.0f} millones).")` Imprime la cuarta conclusión contrastando la región con la tasa porcentual más severa frente a la región que acumula la mayor cantidad de dinero en riesgo en millones.

246 `print(f"5. Provincia mas morosa (cartera >= USD 50 millones): {top_prov.index[0]} ({top_prov['morosidad_pct'].iloc[0]:.2f}%).")` Extrae e imprime de forma automática el nombre y la tasa de la provincia que lidera el ranking crítico de morosidad ponderada representativa.

247 `print(f"6. Las 5 entidades con mas improductiva concentran {pareto_df['acumulado_pct'].iloc[4]:.1f}% del total; la primera es {pareto_df.index[0]}")` Imprime el porcentaje de concentración acumulado en el top 5 bancario y nombra la institución financiera que lidera el volumen nacional en dólares improductivos.

Proyecto final - Dashboard EDA por profesión



248 grandes = seg.index[seg["peso_saldo_pct"] >= 1] Identifica y filtra aquellos segmentos estables del portafolio que representan al menos el 1% del saldo nacional, ignorando carteras marginales para el análisis de variación.

249 sube = cambio.loc[grandes, "var_pp"].sort_values() Toma las variaciones en puntos porcentuales de estos segmentos significativos y los ordena de menor a mayor para evaluar el comportamiento temporal.

250 print(f"7. Variacion ene-ago (segmentos con >= 1% del saldo): mayor baja en {sube.index[0]} ({sube.iloc[0]:+.2f} pp); mayor alza en {sube.index[-1]} ({sube.iloc[-1]:+.2f} pp).") Extrae e imprime de forma automatizada las carteras que presentaron la mejor reducción (baja) y el mayor deterioro (alza) en puntos porcentuales durante el año.

Script preparar_datos.py

Repite exactamente la limpieza del paso 2 en formato script. Sirve para regenerar el CSV sin abrir el notebook: `python preparar_datos.py`. Lee las hojas BASE de data/raw, consolida duplicados, crea variables derivadas y guarda data/processed/cartera_banca_privada_2026.csv.

Dashboard (app.py)

Tabla 2

Bloques de app.py

Bloque de app.py	Qué hace
cargar_datos() @st.cache data	con Lee el CSV una sola vez y lo guarda en memoria para que el tablero responda rápido al cambiar filtros.
tasa() y agrupar()	Funciones reutilizables: calculan la morosidad ponderada (suma/suma) y agrupan por cualquier variable.
Filtros en la barra lateral	Periodo, segmento y región filtran los datos. También están las reglas de decisión: umbral de alerta, umbral de oportunidad y cartera mínima.
Cuatro KPIs	Cartera total, cartera improductiva, morosidad ponderada (con variación en puntos porcentuales frente al mes inicial) y segmento más moroso.
Pestaña Resumen	Línea de morosidad por segmento en el tiempo y barras de dólares improductivos por segmento.
Pestaña Geografía	Mapa de calor segmento x región y ranking de provincias más morosas.
Pestaña Entidades	Matriz de riesgo (burbujas) y concentración de la cartera improductiva.
Pestaña Decisiones	Cruza segmento x región y aplica las reglas: alertas (morosidad >= umbral) y oportunidades (morosidad baja y cartera creciendo). Incluye monitoreo de la Costa por El Niño.
Pestaña Datos	Tabla de datos filtrados y botón para descargar el CSV.

Proyecto final - Dashboard EDA por profesión



3 Parte metodológica: EDA en 7 pasos

El EDA se aplicó como estándar de trabajo. La tabla resume cada paso, qué se hizo y por qué se eligieron esas variables, gráficos y segmentaciones.

Tabla 3

Pasos del EDA

Paso	Qué se hizo	Por qué
1 Pregunta	Se definió la decisión (provisiones y colocación), tres hipótesis y la métrica de morosidad ponderada.	Sin una pregunta clara el EDA se vuelve una colección de gráficos sin destino.
2 Estructura	Carga de 4 Excel, unión, tipos de datos, diccionario, controles de cuadre y duplicados, variables derivadas.	Se verifica que los datos sean confiables antes de analizarlos: por vencer + no devenga + vencida cuadra con el saldo total.
3 Univariado	Conteos, descriptivos, histogramas (saldo en escala log), peso de cada segmento, entidades más grandes.	Entender la forma de cada variable: el saldo está muy sesgado y la morosidad tiene picos en 0% y 100%.
4 Bivariado	Morosidad por segmento en el tiempo, mapa segmento x región, saldo vs morosidad, boxplot.	Buscar relaciones entre variables relevantes para la decisión. El gráfico de saldo vs morosidad muestra un embudo que anticipa el problema de los saldos mínimos.
5 Faltantes y atípicos	Nulos, IQR, cuantificación de filas con saldo menor a USD 100 mil, antes y después.	Los atípicos son de tamaño, no de riesgo. Se decide qué hacer y se documenta.
6 Segmentación	Por segmento, región, provincia, entidad (matriz de riesgo y Pareto) y cantón.	El riesgo es desigual. Segmentar permite decidir dónde actuar primero.
7 Hallazgos	Cifras calculadas por código, lectura de hipótesis, decisiones y limitaciones.	Cierra el ciclo: de los datos a la decisión.

Tratamiento de atípicos (decisión metodológica)

Los indicadores agregados se calculan ponderados con todos los datos: los saldos diminutos casi no pesan, así que no distorsionan.

Los rankings a nivel cantón-entidad usan solo filas con saldo mínimo de USD 100.000. No se imputa ningún valor porque no hay nulos reales.

Se consolidaron 13 duplicados de BP Pichincha en Quito (una fila principal y otra de USD 1) sumando los montos.

Elección de variables y gráficos

Morosidad ponderada: refleja el riesgo real de la cartera y evita el error de promediar porcentajes.

Segmento, región y entidad: son las tres dimensiones sobre las que un analista de riesgo puede actuar.

Escala logarítmica en saldo: el saldo va de pocos dólares a miles de millones; en escala lineal no se vería

Proyecto final - Dashboard EDA por profesión

nada.

Mapa de calor: permite cruzar dos categorías con una métrica numérica en un solo vistazo.

Burbujas por entidad: combinan tamaño, tasa y dólares improductivos en un solo gráfico.

4 Dashboard y decisiones

Cifras del último corte (2026-08). La morosidad total de la banca privada es 3.10% (en 2026-01 fue 3.17%). La cartera improductiva suma USD 1,711 millones sobre USD 55.2 mil millones de cartera.

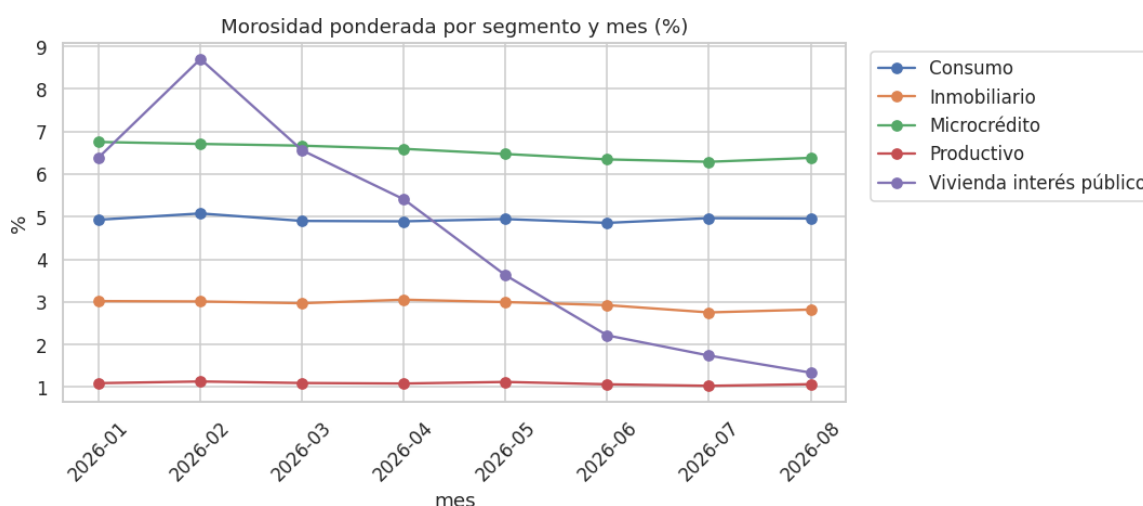
Tabla 4

Descripción por segmento

Segmento	Morosidad	Improductiva	Cartera
Microcrédito	6.38%	USD 250 M	USD 3.92 mil M
Consumo	4.96%	USD 1,100 M	USD 22.18 mil M
Inmobiliario	2.82%	USD 81 M	USD 2.86 mil M
Vivienda interés público	1.34%	USD 1 M	USD 0.07 mil M
Productivo	1.07%	USD 280 M	USD 26.15 mil M

Gráfico 1

Morosidad ponderada por segmento



Proyecto final - Dashboard EDA por profesión

AGUILAR PERALTA J. GABRIEL - GAMBOA PROAÑO MARCO - MEJÍA PARRA MARTHA SUSANA



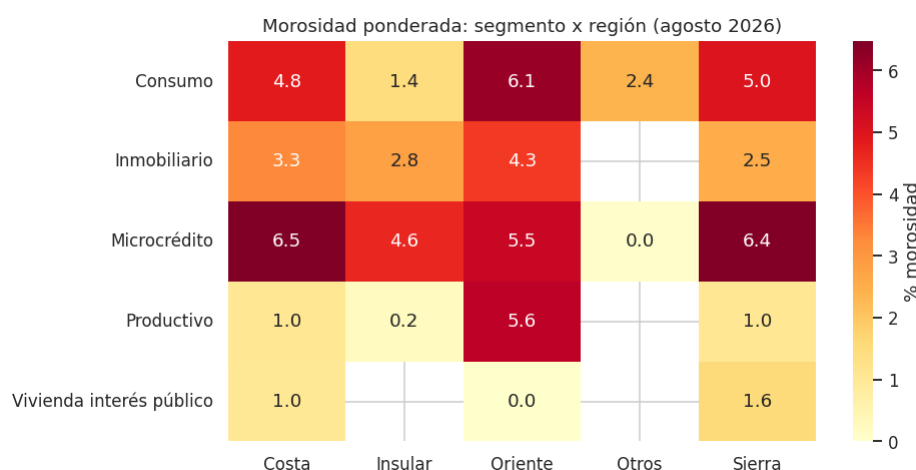
Tabla 5

Descripción por región

Región	Morosidad	Improductiva	Cartera
Oriente	5.65%	USD 34 M	USD 0.60 mil M
Sierra	3.43%	USD 1,058 M	USD 30.84 mil M
Costa	2.61%	USD 618 M	USD 23.67 mil M
Otros	2.38%	USD 0 M	USD 0.00 mil M
Insular	1.83%	USD 1 M	USD 0.07 mil M

Gráfico 2

Morosidad ponderada por segmento por región

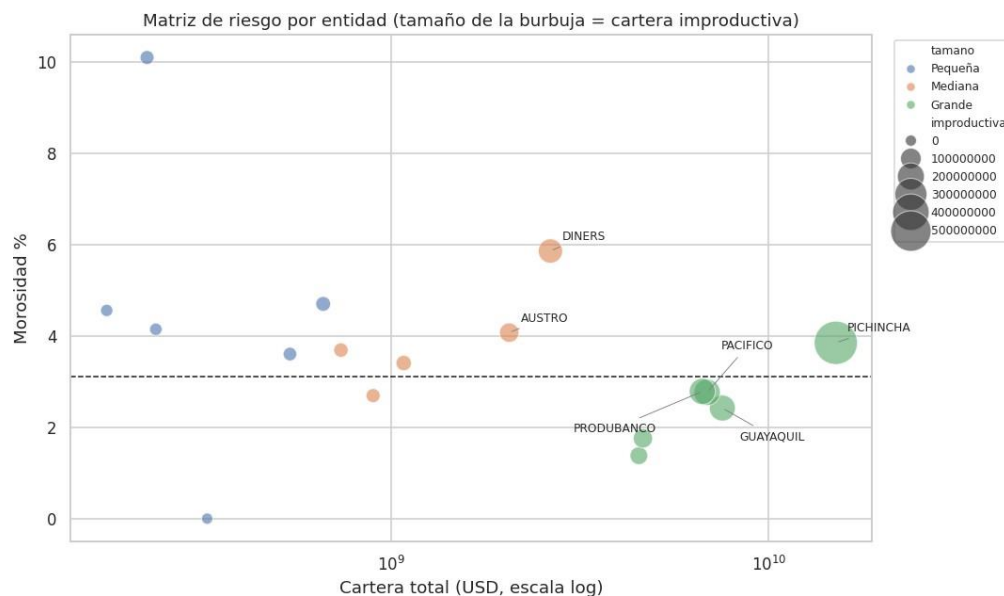


Descripción por entidad

Las cinco entidades con más cartera improductiva concentran 75.5% del total. La primera es PICHINCHA (USD 579 millones), en gran parte por su tamaño.

Gráfico 3

Descripción riesgo por entidad



Hallazgos y decisiones

- **Provisiones:** Microcrédito tiene la tasa más alta (6.38%), pero Consumo concentra 64.3% de los dólares improductivos. La prioridad de provisiones debe mirar ambos.
- **Región:** la Costa (2.61%) no es más morosa que la Sierra (3.43%). El Oriente tiene la tasa más alta (5.65%) pero con muy pocos dólares.
- **Colocación:** Productivo (1.07%) es el segmento más sano y grande; es candidato a ampliar cartera, revisando región por región en el dashboard.
- **Monitoreo de El Niño:** los datos llegan hasta agosto, antes de cualquier efecto climático. La pestaña Decisiones muestra la evolución de la Costa como línea base para detectar deterioro desde septiembre.
- **Concentración:** 75.5% de la improductiva está en 5 entidades; el análisis de riesgo debe empezar por ellas.

Limitaciones

- Solo banca privada: no incluye cooperativas ni banca pública.
- No hay datos de provisiones, por lo que no se calcula la cobertura.

Proyecto final - Dashboard EDA por profesión



- Ocho meses de historia: sirve para tendencia corta, no para estacionalidad anual.
- La morosidad es una definición propia (improductiva / saldo) y puede diferir levemente de la cifra oficial publicada.
- Las asociaciones entre variables no prueban causalidad.

Reproducibilidad

Enlace 1:

<https://github.com/gabo33mh02-u/PROYECTO-FINAL-dashboard-eda-riesgo-credito..git>

Enlace 2:

<https://proyecto-final-dashboard-eda-riesgo-credito-k2uwqjbjrvvfudphu6.streamlit.app/>

Fuentes

- Superintendencia de Bancos del Ecuador, Portal Estadístico, Cartera y Depósitos, Año 2026, banca privada (corte agosto 2026). Contexto sobre morosidad y El Niño: prensa económica ecuatoriana, agosto de 2026.